

Voice Control Color Block Tracking

1. Function Description

Say "Hi, yahboom" to the voice recognition module, and the speaker will announce "I am here". Then say "Track and clip color blocks" to the voice module. At this time, the robotic arm will track the handheld color block. Slowly move the color block, and the robotic arm will correspondingly move along.

Note: When tracking an object, it needs to be moved slowly within the camera's field of view. This case only has tracking function, no grasping function!

2. Startup and Operation

```
# Start camera and inverse kinematics
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start voice recognition and announcement
ros2 launch yahboom_speech speech.launch.py
# Start robotic arm control
ros2 run dofbot_voice_ctrl grasp_VC
# Start color block tracking
ros2 run dofbot_voice_ctrl color_follow_2D
```

3. Core Code Analysis

Color block tracking node color_follow_2D

Source code path:

```
/home/yahboom/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/Color/color_follow_2D.py
```

Parameter initialization

```
def __init__(self, name):

    super().__init__(name)

    # Create an instance of the robotic arm device class
    # 创建机械臂设备类的实例
    self.Arm = Arm_Device()

    # Define initial joint angles for the robot arm
    # 定义机械臂的初始关节角度
    self.init_joints = [90, 150, 0, 20, 90, 0]

    # Bridge for converting between ROS2 Image messages and OpenCV images
    # 用于在ROS2 Image消息和OpenCV图像之间转换的桥接器
    self.rgb_bridge = CvBridge()

    # Flag indicating whether to publish position information
    # 标志位，表示是否发布位置信息
    self.pub_pos_flag = False
```

```
# Current target color
# 当前目标颜色
self.target_color = 0

# File paths for HSV color threshold configuration files
# HSV颜色阈值配置文件的路径
self.red_hsv_text = os.path.join(package_pwd, 'red_colorHSV.text')
self.green_hsv_text = os.path.join(package_pwd, 'green_colorHSV.text')
self.blue_hsv_text = os.path.join(package_pwd, 'blue_colorHSV.text')
self.yellow_hsv_text = os.path.join(package_pwd, 'yellow_colorHSV.text')

# Tuple to store the current HSV range for color filtering
# 存储当前颜色过滤的HSV范围的元组
self.hsv_range = ()

# Flag for ROI (Region of Interest) selection state
# ROI区域选择状态的标志
self.select_flags = False

# Name of the display window for visualization
# 可视化显示窗口名称
self.windows_name = 'frame'

# State machine state for tracking
# 跟踪的状态机状态
self.Track_state = 'init'

# Mouse click position coordinates
# 鼠标点击位置坐标
self.Mouse_XY = (0, 0)

# Image dimensions (columns, rows)
# 图像尺寸（列，行）
self.cols, self.rows = 0, 0

# Initial ROI for color detection (x1, y1, x2, y2)
# 颜色检测的初始感兴趣区域（x1, y1, x2, y2）
self.Roi_init = ()

# Instance of the color detection class
# 颜色检测类的实例
self.color = color_detect()

# Currently selected color name
# 当前颜色名称
self.cur_color = None

# Text color for display
# 显示的文本颜色
self.text_color = (0, 0, 0)

# Center coordinates of the detected object
# 检测到的物体中心坐标
self.cx = 0
self.cy = 0

# Radius of the detected circular object
# 检测到的圆形物体的半径
```

```

self.circle_r = 0

# Message for publishing joint angle
# 发布关节角度的消息
self.joint5 = Int16()
self.corners = np.empty((4, 2), dtype=np.int32)

# Current target color index
# 当前目标颜色索引
self.cur_target_color = 0

# Flag for updating HSV thresholds
# 更新HSV阈值的标志
self.updata_flag = False

# Move the arm to the initial position on startup
# 将机械臂移动到初始位置
self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)

# Subscription to receive raw camera images
# 订阅以接收原始相机图像
self.subscription = self.create_subscription(
    Image, '/image_raw', self.ImageCallback, 1)

# Subscription to receive voice recognition results
# 订阅以接收语音识别结果
self.sub_voice = self.create_subscription(
    Int8, "voice_result", self.getVoiceResultCallBack, 1)

# Variables for storing object position
# 存储物体位置
self.px = 0.0
self.py = 0.0

# Target servo positions for X and Y axes
# X轴和Y轴的目标舵机位置
self.target_servox = 90
self.target_servoy = 45

# PID controllers for X and Y axis servo control
# X轴和Y轴舵机控制的PID控制器
self.xservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)
self.yservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)

# Flags for out-of-range conditions
# 超出范围条件的标志
self.y_out_range = False
self.x_out_range = False

# Current joint angles of the arm
# 机械臂当前的关节角度
self.cur_joints = self.init_joints

# Auxiliary flags for tracking logic
# 跟踪逻辑的辅助标志
self.a = 0
self.b = 0

```

```

# Flag to start the tracking process
# 开始跟踪过程的标志
self.start_flag = False

```

Subscribe to and process voice recognition result topics

```

# Create subscriber for voice recognition result topic
# 创建订阅语音识别结果话题的订阅者
self.sub_voice =
self.create_subscription(Int8,"voice_result",self.getVoiceResultCallback,1)
# Voice recognition callback function
# 语音识别的回调函数
def getVoiceResultCallback(self,msg):
    if msg.data == 106:
        # Voice triggered tracking
        # 语音触发追踪
        self.start_flag = True
        print("Start tracking.")
    elif msg.data == 76:
        self.start_flag = False
        self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
        self.Reset()

```

Color block tracking part

```

def XY_track(self, center_x, center_y):
    # Store the current detected object's center position
    # 存储当前检测到的物体中心位置
    self.px = center_x
    self.py = center_y

    # X-axis Servo Control
    # X轴舵机控制
    if not (self.target_servox >= 180 and center_x <= 320 and self.a == 1 or
            self.target_servox <= 0 and center_x >= 320 and self.a == 1):
        # Normal tracking mode
        # 正常跟踪模式
        if self.a == 0:
            # Set the current object's x-coordinate as the PID system output
            # 将当前物体的x坐标设为PID系统输出
            self.xservo_pid.SystemOutput = center_x

            # Handle out-of-range situation
            # 处理超出范围的情况
            if self.x_out_range == True:
                # Force the servo to the boundary position
                # 强制舵机到达边界位置
                if self.target_servox < 0:
                    self.target_servox = 0
                    self.xservo_pid.SetStepSignal(630)
                if self.target_servox > 0:
                    self.target_servox = 180
                    self.xservo_pid.SetStepSignal(10)
                self.x_out_range = False
            else:

```

```

        # set the step signal to the image center x (320)
        # 将步进信号设为图像中心x坐标 (320)
        self.xservo_pid.SetStepSignal(320)
        self.x_out_range = False

    # Set the inertia time parameters for the PID controller
    # 设置PID控制器的惯性时间参数
    self.xservo_pid.SetInertiaTime(0.01, 0.1)

    # Calculate the target value for the servo
    # 计算舵机的目标值
    target_valuex = int(1500 + self.xservo_pid.SystemOutput)

    # Convert the target value to a servo angle
    # 将目标值转换为舵机角度
    self.target_servox = int((target_valuex - 500) / 10) - 10

    # Check if the calculated angle exceeds the valid range
    # 检查计算出的角度是否超出有效范围
    if self.target_servox > 180:
        self.x_out_range = True
    if self.target_servox < 0:
        self.x_out_range = True

# Y-axis Servo Control
# Y轴舵机控制

if not (self.target_servoy >= 180 and center_y <= 240 and self.b == 1 or
        self.target_servoy <= 0 and center_y >= 240 and self.b == 1):
    if self.b == 0:
        # Set the current object's y-coordinate as the PID system output
        # 将当前物体的y坐标设为PID系统输出
        self.y servo_pid.SystemOutput = center_y

    # Handle out-of-range situation
    # 处理超出范围的情况
    if self.y_out_range == True:
        self.y servo_pid.SetStepSignal(450)
        self.y_out_range = False
    else:
        # set the step signal to the image center y (240)
        # 将步进信号设为图像中心y坐标 (240)
        self.y servo_pid.SetStepSignal(240)

    # Set the inertia time parameters for the PID controller
    # 设置PID控制器的惯性时间参数
    self.y servo_pid.SetInertiaTime(0.01, 0.1)

    # Calculate the target value for the servo
    # 计算舵机的目标值
    target_valuey = int(1500 + self.y servo_pid.SystemOutput)

    # Enforce a minimum value to prevent the arm from going too low
    # 强制执行最小值，防止机械臂降得太低
    if target_valuey <= 1000:
        target_valuey = 1000
        self.y_out_range = True

```

```

# Convert the target value to a servo angle with an offset
# 将目标值转换为带偏移量的舵机角度
self.target_servoy = int((target_valuey - 500) / 10) - 55

# Clamp the angle to the valid range [0, 180]
# 将角度限制在有效范围[0, 180]内
if self.target_servoy > 180:
    self.target_servoy = 180
if self.target_servoy < 0:
    self.target_servoy = 0

# Map servo angle to physical joint angle
# 舵机角度映射到物理关节角度
joint2 = 120 + self.target_servoy
joint3 = self.target_servoy / 4.5
joint4 = self.target_servoy / 3

joints_0 = [float(self.target_servox / 1), float(joint2), float(joint3),
float(joint4), 90.0, 30.0]

# Send the joint commands to the robotic arm
# 向机械臂发送关节控制命令
self.Arm.Arm_serial_servo_write6_array(joints_0, 2000)

# Update the current joint state
# 更新当前关节状态
self.cur_joints = joints_0

```